Round 1

Chitkara Full Stack Engineering Challenge

Preferred Stack: Node.js / JavaScript

Objective

Build and host a REST API (`POST /bfhl`) that accepts an array of node strings, processes hierarchical relationships, and returns structured insights. Also build a frontend that lets users interact with the API.

API Specification

Endpoint

```
POST /bfhl
Content-Type: application/json
```

Request Body

```
{
  "data": ["A->B", "A->C", "B->D"]
}
```

Response Schema

Field	Type	Description
`user_id`	`string`	Format: "fullname_ddmmyyyy" e.g. "johndoe_17091999"
`email_id`	`string`	Your college email address
`college_roll_number`	`string`	Your college roll number
`hierarchies`	`array`	Array of hierarchy objects (see Processing Rules)
`invalid_entries`	`string[]`	Entries that did not match the valid node format
`duplicate_edges`	`string[]`	Repeated edges after the first occurrence
`summary`	`object`	total_trees, total_cycles, largest_tree_root

Hierarchy Object Structure

```
{
  "root":     string,         // root node label
  "tree":     object,         // nested tree (empty {} if cycle)
  "depth"?:   number,         // node count on longest root-to-leaf path
  "has_cycle"?: true          // present only when a cycle is detected
}
```

Summary Object Structure

```
{
  "total_trees":        number,   // count of valid non-cyclic trees
  "total_cycles":       number,   // count of cyclic groups
  "largest_tree_root":  string    // root of tree with greatest depth
}
```

Processing Rules

1. Identity Fields

- `user_id` format: `fullname_ddmmyyyy` (e.g. johndoe_17091999)
- email_id and college_roll_number must be your actual credentials.

2. Valid Node Format

Each valid entry must follow the pattern `X->Y` where X and Y are each a single uppercase letter (A–Z).

The following inputs are invalid and must be pushed to `invalid_entries`:

Input	Reason
`"hello"`	Not a node format
`"1->2"`	Not uppercase letters
`"AB->C"`	Multi-character parent
`"A-B"`	Wrong separator
`"A->"`	Missing child node
`"A->A"`	Self-loop — treated as invalid
`""`	Empty string
`" A->B "`	Trim whitespace first, then validate

3. Duplicate Edges

- If the same Parent->Child pair appears more than once, use the first occurrence for tree construction.
- Push all subsequent occurrences to `duplicate_edges` once each, regardless of how many times they repeat.

Example: `["A->B", "A->B", "A->B"]` → duplicate_edges: `["A->B"]`

4. Tree Construction

- Build trees from valid, non-duplicate edges.
- There can be multiple independent trees - return each separately in the hierarchies array.
- A root is a node that never appears as a child in any valid edge.

- If a group has no valid root (pure cycle - all nodes appear as children), use the lexicographically smallest node as the root.
- Diamond / multi-parent case: if a node has more than one parent (e.g. A->D and B->D), the first-encountered parent edge wins; subsequent parent edges for that child are silently discarded.

5. Cycle Detection

- If a cycle exists within a group, return `has_cycle: true` and `tree: {}`.
- Do not include a `depth` field for cyclic groups.
- For non-cyclic trees, omit `has_cycle` entirely (do not return it as false).

6. Depth Calculation

- Depth = number of nodes on the longest root-to-leaf path.

Example: `A->B->C` → depth: 3 (nodes A, B, C)

7. Summary Rules

- `largest_tree_root` tiebreaker: if two trees have equal depth, return the lexicographically smaller root.
- `total_trees` counts only valid, non-cyclic trees.

Example

Request

```
{
  "data": [
    "A->B", "A->C", "B->D", "C->E", "E->F",
    "X->Y", "Y->Z", "Z->X",
    "P->Q", "Q->R",
    "G->H", "G->H", "G->I",
    "hello", "1->2", "A->"
  ]
}
```

Expected Response

```
{
  "user_id": "johndoe_17091999",
  "email_id": "john.doe@college.edu",
  "college_roll_number": "21CS1001",
  "hierarchies": [
    {
      "root": "A",
      "tree": { "A": { "B": { "D": {} }, "C": { "E": { "F": {} } } } },
      "depth": 4
    },
    {
      "root": "X",
      "tree": {},
      "has_cycle": true
    },
    {
      "root": "P",
```

```
      "tree": { "P": { "Q": { "R": {} } } },
      "depth": 3
    },
    {
      "root": "G",
      "tree": { "G": { "H": {}, "I": {} } },
      "depth": 2
    }
  ],
  "invalid_entries": ["hello", "1->2", "A->"],
  "duplicate_edges": ["G->H"],
  "summary": {
    "total_trees": 3,
    "total_cycles": 1,
    "largest_tree_root": "A"
  }
}
```

Frontend Requirements

Build a single-page frontend that:

- Has an input field or text area where users can enter the node list.
- Has a Submit button that calls your hosted API at /bfhl.
- Displays the API response in a readable, structured format - tree view, cards, or table - your choice. (good looking UI often attracts evaluator ☺).
- Shows a clear error message if the API call fails.

No specific framework is required. Plain HTML/CSS/JS, React, Vue, Next - anything works.

Tech Stack & Hosting

Preferred Stack	Node.js / JavaScript (including frameworks like Next.js, NestJS, Express)
Hosting	Vercel, Render, Netlify, Railway, or any provider of your choice
Code Repository	Push to a public GitHub repository - submission requires the repo URL

Evaluation Notes

- Your API must respond in under 3 seconds for inputs of up to 50 nodes.
- Enable CORS on your API - the evaluator calls it from a different origin.
- The /bfhl route must accept POST requests with Content-Type: application/json.
- Do not hardcode responses - your API will be tested against multiple hidden inputs.
- **Plagiarism check will be run on all GitHub repos.** Identical or near-identical code across submissions = disqualification.

Submission

Fill in the submission form at the link provided to you. You will need to share:

1. Your hosted API base URL (evaluator will call <your-url>/bfhl)
2. Your hosted frontend URL
3. Your public GitHub repository URL

> **Note:** Fastest valid submissions are given priority in the event of tied scores.